# Memory Slice (.msl)

## Binary Format Specification

Version: draft-2026-03
Status: Working Draft

**Abstract**

Memory Slice (`.msl`) is a self-describing, block-based binary format for capturing the forensic state of a single operating system process. It records both the virtual address space (with per-page acquisition status) and transient OS-queryable metadata that exists only while the process is alive. This document specifies the binary layout: file header, block architecture, integrity chain, capture-time payloads, cross-referencing, capability bitmap, and raw dump import mechanism.

This specification accompanies the research paper *Memory Slice: A Process-Centric Dump Format Enabling Differential Structure Discovery in Memory Forensics*. The differential analysis algorithm, MemDiver framework, and evaluation are outside this document's scope.

# Contents


**1 Scope and Conventions** **2**
- 1.1 Scope . . . 2
- 1.2 Normative Language . . . 2
- 1.3 Encoding Conventions . . . 2
- 1.4 UUID Requirements . . . 2
- 1.5 Terminology . . . 2

**2 Format Overview** **4**
- 2.1 Design Principles . . . 4

**3 File Header** **4**
- 3.1 Endianness Bootstrapping . . . 5
- 3.2 Version Compatibility . . . 5

**4 Block Architecture** **7**
- 4.1 Common Block Header (80 bytes) . . . 7
- 4.2 Block Flags . . . 7
- 4.3 Block Type Registry . . . 8
- 4.4 Integrity Chain (`PrevHash`) . . . 8
- 4.5 End-of-Capture Block (`0x0FFF`) . . . 9

**5 Capture-Time Payloads** **10**
- 5.1 Memory Region (`0x0001`) . . . 10
- 5.2 Module Entry (`0x0002`) . . . 10

**6 Three-State Virtual Address Map** **11**

**7 Block Cross-Referencing** **12**

**8 Capability Bitmap** **13**

**9 Raw Dump Import** **14**
- 9.1 Import Provenance Block (`0x0030`) . . . 14
- 9.2 Import Procedure . . . 14

**10 Parsing Walkthrough** **16**

**11 Worked Example** **17**

**12 Conformance** **18**
- 12.1 Producer . . . 18
- 12.2 Consumer . . . 18

# 1 Scope and Conventions

## 1.1 Scope

This document defines the binary wire format of Memory Slice (`.msl`) files. It does *not* specify acquisition procedures or analysis algorithms.

## 1.2 Normative Language

The key words **MUST**, **MUST NOT**, **REQUIRED**, **SHALL**, **SHALL NOT**, **SHOULD**, **SHOULD NOT**, "RECOMMENDED", **MAY**, and **OPTIONAL** are per RFC 2119 [1].

## 1.3 Encoding Conventions

- The file header's `Endianness` byte at offset `0x08` determines the byte order for all multi-byte integers throughout the file. `0x01` = little-endian (default); `0x02` = big-endian. Producers **SHOULD** use little-endian unless the target platform is natively big-endian. Because this is a single `uint8`, it has no endianness ambiguity (Section 3.1).
- Strings are UTF-8, null-terminated, padded to **8-byte** boundaries with zero bytes. $\text{pad8}(n) = \lceil n/8 \rceil \times 8$. All padding bytes inserted to reach alignment boundaries **MUST** be initialized to `0x00`. This prevents tools from leaking sensitive heap contents into padding space.
- Variable-length fields (`PageStateMap`, `NativeBlob`) are padded to 8-byte boundaries with zero bytes.
- UUIDs **MUST** be version 4 (random) per RFC 4122 [2], stored as 16 raw bytes in RFC 4122 network byte order (big-endian sub-fields), regardless of the file's `Endianness`. See Section 1.4.
- Hashes use BLAKE3 [3] with 32-byte output (Section 4.4).
- Timestamps: unsigned 64-bit nanoseconds since Unix epoch.
- All `Reserved` fields **MUST** be zero (producers) and **MUST** be ignored (consumers).

## 1.4 UUID Requirements

All UUID fields (`DumpUUID`, `BlockUUID`, `ParentUUID`, payload UUIDs) **MUST** be version 4 (random) per RFC 4122. The 4-bit version field (bits 48–51) **MUST** be `0100`; the 2-bit variant (bits 64–65) **MUST** be `10`. UUIDs are always in RFC 4122 binary layout regardless of the file's endianness.

**Implementation guidance: UUID generation performance**

UUID generation can bottleneck high-throughput acquisition. Producers need not use a CSRNG; UUIDs are identifiers, not security tokens. A thread-local fast PRNG such as `xoshiro256++` or `PCG64`, seeded once from OS entropy, provides sufficient uniqueness ($\sim 2^{-61}$ collision probability per pair) while avoiding CSRNG overhead per block.

## 1.5 Terminology

**Producer**
: Software that creates or appends blocks.

**Consumer**
Software that reads an MSL file.

**Acquirer**
A producer capturing live process state (types `0x0000`–`0x0FFF`).

**Importer**
A producer converting a raw dump into MSL (Section 9).

**Block**
Fixed header + variable payload.

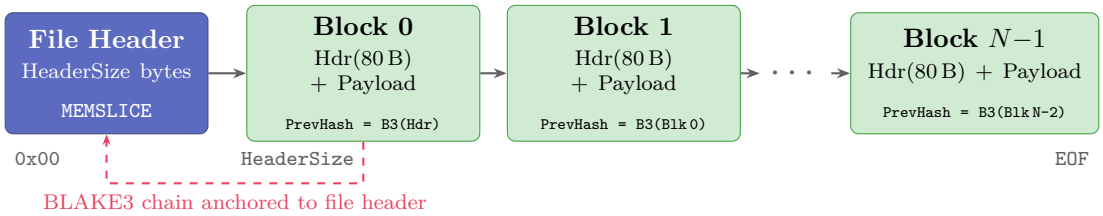
**Payload**
Block-type-specific data after the header.

---

*The preceding section established the conventions, terminology, and encoding rules that apply throughout the specification. The following sections define the format itself, starting with the high-level file structure and then drilling into each component: the file header (Section 3), block architecture (Section 4), capture-time payloads (Section 5), the three-state page model (Section 6), cross-referencing (Section 7), and the capability bitmap (Section 8). Sections 9–12 cover raw dump import, parsing guidance, a worked example, and conformance requirements.*

# 2 Format Overview

An MSL file is a linear byte stream: a **file header** (64 bytes in v1.0) followed by **typed, length-prefixed blocks**. No inter-block gaps. `HeaderSize` tells the consumer where blocks begin, enabling future header extensions.




Block 0's `PrevHash` = $BLAKE3$(File Header), anchoring the integrity chain. Each subsequent block hashes its predecessor, so modifying any block in the middle invalidates all subsequent hashes. Consumers detect end-of-file when fewer than 80 bytes remain or the next 4 bytes do not match the block magic (`0x4D534C43`).

**Integrity of the last block**

The forward-linked BLAKE3 chain protects every block except the last one, since no subsequent block hashes it. To close the integrity loop within the file, acquirers **SHOULD** emit an End-of-Capture (EoC) block (type `0x0FFF`, Section 4.5) as the final capture-time block. The EoC contains a BLAKE3 digest of the entire file up to that point, enabling full integrity verification without external systems. If no EoC is present (e.g., the acquirer crashed), consumers **SHOULD** compute $BLAKE3$(entire file) and verify against an externally stored digest.

## 2.1 Design Principles

1. **Forensic Integrity (Append-Only, Anchored Chain).** Immutable capture blocks. BLAKE3 chain from file header through all blocks.
2. **Epistemic Honesty (Three-State Pages).** Captured vs. Failed vs. Unmapped.
3. **Self-Describing.** Endianness, version, header size, OS, arch, PID, capabilities.
4. **Dual-Layer OS Abstraction.** Normalized fields + OS-native opaque blob.

**No block count in the header**

The header has no block count. Appending blocks would require mutating it, violating append-only integrity. Consumers read blocks sequentially until fewer than 80 bytes remain or the next 4 bytes do not match the block magic.

# 3 File Header

The header (Table 1) places the single-byte `Endianness` at offset `0x08` (after the 8-byte magic) and the single-byte `HeaderSize` at `0x09`. A parser reads these two `uint8` values—which have no byte-order ambiguity—before encountering any multi-byte field.

Table 1: File header (64 bytes in version 1.0).

| Offset | Size | Field | Description |
|---|---|---|---|
| 0x00 | 8 | Magic | 0x4D454D534C494345 ("MEMSLICE"). Endianness-independent byte sequence. |
| 0x08 | 1 | Endianness | 0x01=LE, 0x02=BE. Single byte: no endianness ambiguity. Invalid values **MUST** cause rejection. |
| 0x09 | 1 | HeaderSize | Header size in bytes (uint8). v1.0: **MUST** be 64 (0x40). First block starts here. |
| 0x0A | 2 | Version | High byte=major, low byte=minor. v1.0: 0x0100. Read per Endianness. |
| 0x0C | 4 | Flags | Bit 0: Imported (raw dump import). Bits 1–31: reserved, zero. |
| 0x10 | 8 | CapBitmap | Capability bitmap (Section 8). |
| 0x18 | 16 | DumpUUID | UUIDv4 for this dump. |
| 0x28 | 8 | Timestamp | Acquisition start (UTC, ns since epoch). |
| 0x30 | 2 | OSType | Target OS (Table 2a). |
| 0x32 | 2 | ArchType | Target CPU (Table 2b). |
| 0x34 | 4 | PID | Process ID at acquisition. |
| 0x38 | 8 | Reserved | **MUST** be zero. |

**Size:** $8 + 1 + 1 + 2 + 4 + 8 + 16 + 8 + 2 + 2 + 4 + 8 = 64$.

### 3.1 Endianness Bootstrapping

This uses the ELF approach (e_ident[EI_DATA] [4]): a single byte whose value is always read identically regardless of host byte order.
**Parse sequence:** (1) Read 8-byte magic. (2) Read Endianness at 0x08. (3) Read HeaderSize at 0x09. (4) All subsequent multi-byte reads use the determined order.

Memory content within PageData is stored in the *target architecture's* native order; Endianness governs only format-level structural integers.

### 3.2 Version Compatibility

Unknown major version: **SHOULD** reject. Unknown minor of known major: **MAY** parse. Always use HeaderSize to locate Block 0.
Table 2 defines the registered codes for the OSType and ArchType header fields.

Table 2: Target platform codes: (a) Operating systems and (b) CPU architectures.

**(a) OS type codes** (`OSType`)

| Code | OS |
|---|---|
| 0x0000 | Windows |
| 0x0001 | Linux |
| 0x0002 | macOS |
| 0x0003 | Android |
| 0x0004 | iOS / iPadOS |
| 0x0005 | FreeBSD |
| 0x0006 | NetBSD |
| 0x0007 | OpenBSD |
| 0x0008 | QNX |
| 0x0009 | Fuchsia |
| 0x000A-0x00FF | *Reserved* |
| 0x0100-0xFFFE | *Vendor* |
| 0xFFFF | Unknown |

**(b) Architecture codes** (`ArchType`)

| Code | Architecture |
|---|---|
| 0x0000 | x86 (IA-32) |
| 0x0001 | x86_64 (AMD64) |
| 0x0002 | ARM64 (AArch64) |
| 0x0003 | ARM32 (AArch32) |
| 0x0004 | MIPS32 |
| 0x0005 | MIPS64 |
| 0x0006 | RISC-V RV32 |
| 0x0007 | RISC-V RV64 |
| 0x0008 | PPC32 |
| 0x0009 | PPC64 |
| 0x000A | s390x |
| 0x000B | LoongArch64 |
| 0x000C-0x00FF | *Reserved* |
| 0x0100-0xFFFE | *Vendor* |
| 0xFFFF | Unknown |

---

*The file header provides the bootstrapping context—endianness, version, capabilities, and the offset at which blocks begin. The next section defines the block structure that makes up the rest of the file: the common header shared by all blocks, the type registry, and the BLAKE3 integrity chain.*

# 4 Block Architecture

Every block in an MSL file consists of a **fixed 80-byte common header** followed by a **variable-length payload**. The common header has the same layout regardless of block type, which enables forward compatibility: a consumer that does not recognize a block's type reads the `BlockLength` field and skips (`BlockLength` − 80) bytes to reach the next block. Consumers detect end-of-file when fewer than 80 bytes remain after the current block or the next 4 bytes do not match the block magic (`0x4D534C43`).

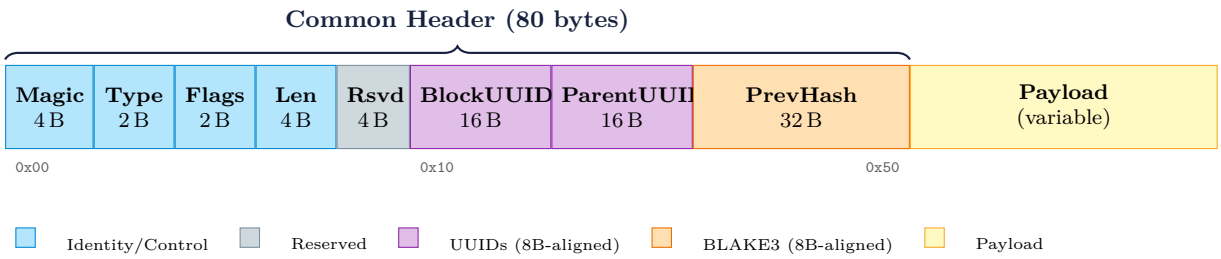
## 4.1 Common Block Header (80 bytes)

The header layout (Table 3) ensures that all fields of 8 bytes or larger start on 8-byte-aligned offsets, enabling zero-copy casting of UUIDs to `uint64_t` pairs and direct BLAKE3 digest comparison without unaligned memory access faults (critical on strict architectures such as ARM).

Table 3: Block header (80 bytes). All fields $\geq 8$ B are 8-byte aligned.

| Off | Size | Field | Description |
|---|---|---|---|
| 0x00 | 4 | Magic | 0x4D534C43 ("MSLC"). |
| 0x04 | 2 | BlockType | Payload selector (Section 4.3). |
| 0x06 | 2 | Flags | Per-block flags (Table 4). |
| 0x08 | 4 | BlockLength | Total size (hdr+payload). $\geq 80$. |
| 0x0C | 4 | Reserved | **MUST** be zero. |
| 0x10 | 16 | BlockUUID | UUIDv4 identifier. *8-byte aligned.* |
| 0x20 | 16 | ParentUUID | Parent UUID or zeros. *8-byte aligned.* |
| 0x30 | 32 | PrevHash | BLAKE3 of preceding element (Section 4.4). *8-byte aligned.* |
| 0x50 | var | Payload | BlockLength − 80 bytes. *8-byte aligned.* |

**Verification:** $4 + 2 + 2 + 4 + 4 + 16 + 16 + 32 = 80$. Payload starts at `0x50`.




## 4.2 Block Flags

The 16-bit `Flags` field at block offset `0x06` carries per-block metadata. The defined flags are listed in Table 4.

Table 4: Block flags (16-bit at `0x06`).

| Bit(s) | Name | Description |
|---|---|---|
| 0 | `Compressed` | Algorithm in bits 1–2. |
| 1–2 | `CompAlgo` | 00=none, 01=zstd, 10=lz4, 11=reserved. |
| 3 | `Encrypted` | Key in Key block (`0x0020`). |
| 4 | `Optional` | **MAY** skip if unrecognized. |
| 5 | `HasChildren` | Referenced via `ParentUUID`. |
| 6–15 | `Reserved` | Zero. |

## 4.3 Block Type Registry

Block types are partitioned into three namespaces (Table 5), separating capture-time data from analysis-produced annotations. This enforces the append-only principle: acquirers write capture-time blocks; analysis tools **MUST** only write Structural or Semantic blocks. Table 6 lists all types defined by this specification.

Table 5: Block type namespaces.

| Range | Namespace | Writer | Examples |
|---|---|---|---|
| `0x0000`–`0x0FFF` | Capture-Time | Acquirer/Importer | Regions, modules, threads, FDs, network, import provenance |
| `0x1000`–`0x1FFF` | Structural | Analysis tool | VAS map, classification, pointer graph |
| `0x2000`–`0x2FFF` | Semantic | Analysis tool | Structure candidates, annotations, signatures |

Table 6: Defined block types.

| Type | Name | Description |
|---|---|---|
| `0x0001` | Memory Region | Per-page memory (Sec. 5.1). |
| `0x0002` | Module Entry | Module metadata (Sec. 5.2). |
| `0x0010` | Module List Index | Groups Module Entries. |
| `0x0011` | Thread Context | Register state. |
| `0x0012` | File Descriptor | Open handle. |
| `0x0013` | Network Connection | Socket attribution. |
| `0x0014` | Environment Block | Env vars. |
| `0x0015` | Security Token | Credentials. |
| `0x0020` | Key Block | Encryption key material. |
| `0x0030` | Import Provenance | Raw dump metadata (Sec. 9). |
| `0x0FFF` | End-of-Capture | Whole-file integrity seal (Sec. 4.5). |
| `0x1001` | VAS Map | Reconstructed address space. |
| `0x1002` | Region Classification | Stability tags. |
| `0x1003` | Pointer Graph | Pointer relationships. |
| `0x2001` | Structure Candidate | Discovered type. |
| `0x2002` | Analyst Annotation | Free-form note. |
| `0x2003` | Diff Result | Differential analysis. |
| `0x2004` | Scan Signature | Byte pattern export. |

## 4.4 Integrity Chain (`PrevHash`)

The chain uses BLAKE3 [3] with 32-byte output. BLAKE3 is typically 3–4× faster than SHA-256 on modern CPUs, SIMD-friendly, and provides equivalent 128-bit collision resistance.

- **Block 0:** `PrevHash` $= BLAKE3$(File Header, all `HeaderSize` bytes).
- **Block $i$ ($i \geq 1$):** `PrevHash` $= BLAKE3$(Block$_{i-1}$, all `BlockLength` bytes).

**Wire-format hashing.** The `PrevHash` is always computed over the *raw on-disk bytes* of the preceding element—the exact byte sequence as stored in the file. If a block's payload is compressed (per the `Compressed` flag), the hash covers the compressed wire format, not the decompressed content. This ensures that integrity verification requires only sequential byte reads without decompression.

**Why hash the previous element?**

A block cannot contain its own hash (circular dependency). Each element commits to its predecessor. The chain is transitively anchored: modifying any block in the middle invalidates all subsequent hashes.

### 4.5 End-of-Capture Block (`0x0FFF`)

The forward-linked hash chain has an inherent limitation: the last block in the file is not hashed by any successor. To close the integrity loop *within the file itself*, an acquirer **SHOULD** emit an End-of-Capture (EoC) block as the final block after all other capture-time blocks have been written. The EoC payload (Table 7) contains a single BLAKE3 digest of the entire file up to (but not including) the EoC block itself.

Table 7: End-of-Capture payload.

| Off | Size | Field | Description |
|---|---|---|---|
| +0x00 | 32 | `FileHash` | BLAKE3 digest of all bytes from file offset 0 through the last byte before this EoC block. |
| +0x20 | 8 | `AcqEnd` | Acquisition end time (UTC, ns since epoch). |
| +0x28 | 8 | `Reserved` | **MUST** be zero. |

**Integrity semantics.** If the EoC block is present, a consumer can verify the entire file in two steps: (1) verify the `PrevHash` chain from the file header through all blocks including the EoC block, and (2) compute $BLAKE3$(file bytes $[0 \ldots \text{EoC offset}-1]$) and compare against `FileHash`. Together, these two checks provide complete integrity coverage without any external digest.

If the acquirer crashes during capture, the EoC block is simply absent—alerting the analyst that the dump is incomplete. Analysis tools that append enrichment blocks after the EoC do not invalidate `FileHash`: it covers only the original capture data, and the `PrevHash` chain continues to protect the appended blocks individually.

---

*With the block architecture, type registry, and integrity chain defined, the next two sections specify the actual payload content: how process memory and module metadata are encoded within their respective block types.*

# 5 Capture-Time Payloads

Payload data begins at byte `0x50` within each block (immediately after the 80-byte common header). All field offsets in this section are **relative to the payload start** unless otherwise noted. This section defines the two most critical capture-time payloads: Memory Region blocks, which store the actual virtual address space content, and Module Entry blocks, which store the metadata needed to interpret that content.

## 5.1 Memory Region (`0x0001`)

Each captured memory region is stored as a separate block (Table 8). An acquirer **SHOULD** emit one block per contiguous virtual address range as enumerated by the operating system (e.g., via `VirtualQuery` on Windows or `/proc/pid/maps` on Linux). Because acquisition is not atomic, individual pages within a region may become unreadable or unmapped between enumeration and read time; the `PageStateMap` records this per-page outcome (see Section 6).

Table 8: Memory Region payload.

| Off | Size | Field | Description |
|---|---|---|---|
| +0x00 | 8 | BaseAddr | Region start VA. |
| +0x08 | 8 | RegionSize | Size (multiple of PageSize). |
| +0x10 | 1 | Protection | Bit 0=R, 1=W, 2=X. |
| +0x11 | 1 | RegionType | 0x00=Unknown, 01=Heap, 02=Stack, 03=Image, 04=MappedFile, 05=Anon, 06=SharedMem, FF=Other. |
| +0x12 | 2 | PageSize | Typically 4096. |
| +0x14 | 4 | MapLength | PageStateMap size in bytes. |
| +0x18 | 8 | Timestamp | Acquisition time (ns). |
| +0x20 | var | PageStateMap | 2 bits/page, padded to 8B. |
| $+m$ | var | PageData | Captured pages only. |

**MapLength:** Let $P$ = `RegionSize`/`PageSize`. Integer: `MapLength` = $\text{pad8}((P+3)/4)$, i.e. $\text{pad8}(n) = (n+7)\ \&\ \sim 7$.
**Overhead.** The `PageStateMap` adds minimal size: at 2 bits per page, a 1 GiB region (262,144 pages of 4 KiB) requires only a 64 KiB map—0.006% overhead relative to the page data.

## 5.2 Module Entry (`0x0002`)

Each loaded module (DLL, shared object, dynamic library) is stored as a separate block whose `ParentUUID` **MUST** reference the Module List Index (type `0x0010`). The payload (Table 9) captures both platform-independent fields (path, version, on-disk hash) and an OS-native opaque blob for platform-specific analysis.

Table 9: Module Entry payload.

| Off | Size | Field | Description |
|---|---|---|---|
| +0x00 | 8 | BaseAddr | Load address. |
| +0x08 | 8 | ModuleSize | Mapping size. |
| +0x10 | 2 | PathLen | Exact byte length of Path including the null terminator, *before* padding. The stored (padded) size is pad8(PathLen). |
| +0x12 | 2 | VersionLen | Exact byte length of Version including the null terminator, *before* padding. 0 if version is unavailable (no Version field emitted). The stored (padded) size is pad8(VersionLen). |
| +0x14 | 4 | Reserved | Zero. |
| +0x18 | var | Path | UTF-8, pad8. |
| $+p$ | var | Version | $p =$ 0x18 + pad8(PathLen). |
| $+v$ | 32 | DiskHash | BLAKE3 of on-disk binary. $v = p +$ pad8(VersionLen). |
| $+v$+32 | 4 | BlobLen | NativeBlob size. |
| $+v$+36 | 4 | Reserved2 | Zero. |
| $+v$+40 | var | NativeBlob | OS-native opaque data. |

**Module list as a decoder ring**

The exact version of `ntdll.dll` determines NT Heap chunk layout and XOR key; the exact `libc` determines glibc `malloc` chunk format. Without this, a heap dump is opaque.

## 6 Three-State Virtual Address Map

The `PageStateMap` in each Memory Region block encodes one of three states for every page using 2 bits per page. Table 10 defines the state codes.

Table 10: Page states.

| Bits | State | Meaning | PageData |
|---|---|---|---|
| 00 | Captured | Read OK; stored. | PageSize bytes |
| 01 | Failed | Mapped but unreadable. | 0 |
| 10 | Unmapped | Not present at read time (TOCTOU). | 0 |
| 11 | *Reserved* | Treat as Failed. | 0 |

Only 00 pages contribute to `PageData`.

**Why Unmapped exists within a region**

Acquisition is not atomic: regions are enumerated first, then pages are read. Between enumeration and read, the process may unmap pages (TOCTOU race). `Unmapped` records this: the page was in the region at enumeration but absent at read time—distinct from `Failed` (mapped but unreadable) and from never appearing at all.

**Reconstruction algorithm.** To rebuild the virtual address space from a Memory Region block, a consumer iterates the `PageStateMap` 2 bits at a time, maintaining a cursor into `PageData`:

1. For page index $i = 0, 1, \ldots, (\texttt{RegionSize}/\texttt{PageSize}) - 1$:
   a. Read 2 bits from `PageStateMap` at bit position $2i$.
   b. If `00` (Captured): copy `PageSize` bytes from the cursor into virtual address $\texttt{BaseAddr} + i \times \texttt{PageSize}$; advance cursor by `PageSize`.
   c. If `01` (Failed) or `10` (Unmapped): record the state at this address; do *not* advance the `PageData` cursor.

# 7 Block Cross-Referencing

Blocks in an MSL file do not exist in isolation—they form a graph of relationships that enables consumers to navigate from a discovered structure candidate back to its source memory region and the module whose allocator produced it. The cross-referencing model provides two complementary layers:

**Layer 1: `ParentUUID` (common header).** Every block's header includes a `ParentUUID` field that establishes a "belongs-to" hierarchy. Because this field is part of the common header, any consumer can traverse it without understanding payload layouts. For example, a Module Entry block's `ParentUUID` references the Module List Index block that groups all modules, and a Memory Region block's `ParentUUID` may reference the Module Entry for the library whose image occupies that region.

**Layer 2: Payload-embedded UUIDs.** For relationships that do not fit a simple parent–child model, individual payload layouts include dedicated UUID fields. A Structure Candidate block (type `0x2001`), for instance, contains both a *source-region UUID* identifying the Memory Region where the candidate was found and a *module UUID* identifying the Module Entry whose allocator version informed the discovery. These richer relationships require type-aware parsing.

Figure 1 illustrates both layers. Solid arrows represent `ParentUUID` links traversable by any consumer; dashed arrows represent payload-embedded UUIDs requiring knowledge of the block type's payload layout.

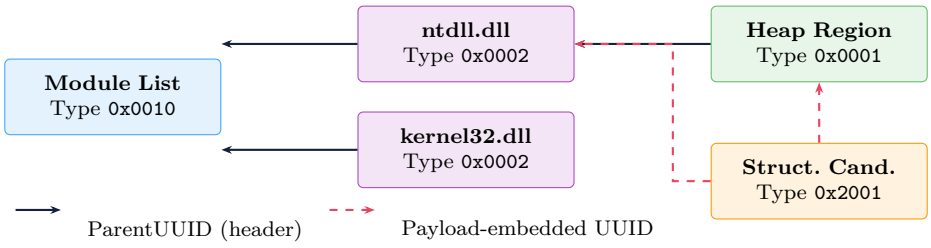



Figure 1: Block cross-referencing example. Two Module Entry blocks (`ntdll.dll`, `kernel32.dll`) reference their Module List Index via `ParentUUID` (solid). A Heap Region references its owning module. A Structure Candidate block references both its source region and the relevant module via payload-embedded UUIDs (dashed), enabling an analyst to trace a discovered structure back to the allocator version that produced it.

# 8 Capability Bitmap

The file header's `CapBitmap` (8 bytes, 64 bits) documents which metadata categories the acquirer was able to capture. A consumer **SHOULD** check this bitmap before searching for specific block types. Table 11 lists the defined bits.

Table 11: Capability bits (64-bit `CapBitmap`).

| Bit | Name | Description |
|---|---|---|
| 0 | MemoryRegions | Type `0x0001` present. |
| 1 | ModuleList | Types `0x0010`/`0x0002` present. |
| 2 | ThreadContexts | Type `0x0011`. |
| 3 | FileDescriptors | Type `0x0012`. |
| 4 | NetworkState | Type `0x0013`. |
| 5 | EnvironmentVars | Type `0x0014`. |
| 6 | SharedMemory | IPC identifiers. |
| 7 | SecurityContext | Type `0x0015`. |
| 8–63 | Reserved | Zero. |

An acquirer **MUST** set exactly those bits corresponding to the metadata categories it successfully captured. Setting a bit without emitting the corresponding blocks, or emitting blocks without setting the bit, are both non-conformant. An importer (Section 9) typically sets only bit 0 (`MemoryRegions`) unless the source format provides additional metadata.

---

*The preceding sections defined the format for live process acquisition—the ideal case where an acquirer captures memory alongside rich OS metadata. In practice, analysts frequently encounter raw process dumps from existing tools that lack this metadata. The next section defines how such dumps can be imported into the MSL format, preserving whatever information the source provides and making the gaps explicit.*

# 9 Raw Dump Import

Raw process dumps (`/proc/pid/mem`, `ReadProcessMemory`, `gcore`, Minidumps) lack forensic metadata. MSL supports importing them into valid MSL files, enabling unified downstream tooling.

## 9.1 Import Provenance Block (`0x0030`)

An importer **MUST** emit exactly one Import Provenance block as the first block (Block 0). The payload (Table 12) records the source format (Table 13), the importing tool's identity, and a free-form note describing any conversion limitations.

Table 12: Import Provenance payload.

| Off | Size | Field | Description |
|---|---|---|---|
| +0x00 | 2 | SourceFormat | See Table 13. |
| +0x02 | 2 | Reserved | Zero. |
| +0x04 | 4 | ToolNameLen | Tool name length (incl. null). |
| +0x08 | 8 | ImportTime | Import time (ns). |
| +0x10 | 8 | OrigFileSize | Original file size. 0 if unknown. |
| +0x18 | 4 | NoteLen | Note length (incl. null). 0 if none. |
| +0x1C | 4 | Reserved2 | Zero. |
| +0x20 | var | ToolName | UTF-8, pad8. |
| $+t$ | var | Note | UTF-8, pad8. |

Table 13: Source format codes.

| Code | Format | Notes |
|---|---|---|
| 0x0000 | Unknown | |
| 0x0001 | Raw byte stream | `/proc/pid/mem`, `ReadProcessMemory`. |
| 0x0002 | ELF core dump | `gcore(1)` or kernel core. |
| 0x0003 | Windows Minidump | `.mdmp`. |
| 0x0004 | macOS core dump | Mach-O core. |
| 0x0005 | ProcDump | Sysinternals. |
| 0x0006-0xFFFE | *Reserved* | |
| 0xFFFF | Other | |

## 9.2 Import Procedure

1. **File header:** Set `Flags` bit 0 (`Imported`). Set `OSType/ArchType/PID` if known, else `0xFFFF/0xFFFF/0`. Set `CapBitmap` to reflect only available metadata.
2. **Block 0:** Import Provenance (`0x0030`).
3. **Memory Regions:** For raw streams without addresses: single region, `BaseAddr`=0, all Captured. For ELF cores: parse program headers. For Minidumps: parse `MINIDUMP_MEMORY_LIST`.
4. **Modules:** Extract from source if available (ELF `NT_FILE`, Minidump module list).
5. **Hash chain and EoC:** Block 0's PrevHash = $BLAKE3$(File Header); subsequent blocks chain normally. Emit an End-of-Capture block (`0x0FFF`) as the final block.

**What import preserves and what it loses**

**Gains:** standardized container, three-state page map, UUID cross-referencing, enrichment capability. **Retains:** whatever the source provided (ELF program headers, Minidump module lists). **Cannot recover:** live FD tables, network attribution, precise module versions (unless in source), IPC identifiers, security tokens. The `CapBitmap` and Import Provenance make these gaps explicit.

---

*The specification has now defined all structural elements: file header, block architecture, integrity chain, payload layouts, page state model, cross-referencing, capability bitmap, and raw dump import. The remaining sections provide practical guidance: a step-by-step parsing walkthrough, a fully worked byte-level example, and the formal conformance requirements for producers and consumers.*

# 10 Parsing Walkthrough

**Step 1: File header.**
Read 9 bytes. Verify magic. `Endianness` at `0x08`. `HeaderSize` at `0x09`. Read remaining header in determined byte order.

**Step 2: Blocks.**
Start at `HeaderSize`. Read 80B header, verify block `Magic` (`0x4D534C43`), parse or skip per `BlockType`/`BlockLength`. Continue until fewer than 80 bytes remain or the next 4 bytes do not match the block magic.

**Step 3: Integrity chain (OPTIONAL but recommended).**
Compute $BLAKE3$(header), verify against Block 0's `PrevHash`. For each block $i \geq 1$, compute $BLAKE3(\text{block}_{i-1})$ over its raw on-disk bytes and compare against block $i$'s `PrevHash`.

**Step 4: EoC verification.**
If an End-of-Capture block (`0x0FFF`) is present, compute $BLAKE3$ over all file bytes preceding it and verify against `FileHash`. If absent, the dump may be incomplete or the acquirer did not emit one; consider external digest verification.

**Step 5: Index.**
`BlockUUID`→offset, `ParentUUID`→children.

**Step 6: Import check.**
`Flags` bit 0 set? Read Import Provenance (`0x0030`).

**Step 7: Payloads.**
Regions via PageStateMap. Modules via pad8 offsets.

# 11 Worked Example

Table 14 shows a complete Module Entry block for `ntdll.dll` version `10.0.22621.1` at `0x7FFDD4D00000`, in a little-endian file.
**Sizes:** Path=30B (pad8=32). Version=13B (pad8=16). Payload: 8+8+2+2+4+32+16+32+4+4+384 = 496B. Block: 80+496 = 576 (`0x0240`).

Table 14: ntdll.dll Module Entry.

| Off | Field | Hex | Value |
|---|---|---|---|
| *— Header (80B) —* | | | |
| 0x00 | Magic | 4D 53 4C 43 | MSLC |
| 0x04 | BlockType | 02 00 | Module Entry |
| 0x08 | BlockLength | 40 02 00 00 | 576 |
| 0x0C | Reserved | 00 00 00 00 | |
| 0x10 | BlockUUID | (16B) | UUIDv4 |
| 0x20 | ParentUUID | (16B) | Module List UUID |
| 0x30 | PrevHash | (32B) | BLAKE3 of prev |
| *— Payload (496B) —* | | | |
| 0x50 | BaseAddr | 00 00 D0 D4 FD 7F 00 00 | 0x7FFDD4D00000 |
| 0x58 | ModuleSize | 00 00 20 00 00 00 00 00 | 2 MiB |
| 0x60 | PathLen | 1E 00 | 30 |
| 0x62 | VersionLen | 0D 00 | 13 |
| 0x68 | Path (32B) | 43 3A 5C 57... | ntdll.dll |
| 0x88 | Version (16B) | 31 30 2E 30... | 10.0.22621.1 |
| 0x98 | DiskHash (32B) | ... | BLAKE3 of binary |
| 0xB8 | BlobLen | 80 01 00 00 | 384 |
| 0xC0 | NativeBlob | (384B) | LDR_DATA_TABLE_ENTRY |

*The worked example above demonstrates how the specification's rules compose into a concrete block. The final section formalizes the obligations of producers and consumers, distinguishing mandatory requirements from recommendations.*

# 12 Conformance

## 12.1 Producer

A conformant producer **MUST**: (1) write a valid header with `Magic`, `Endianness` (default `0x01`), `HeaderSize`, `Version`; (2) use byte order per `Endianness`; (3) generate UUIDv4 in RFC 4122 network byte order; (4) compute BLAKE3 `PrevHash`: Block 0 hashes the header, subsequent blocks hash the preceding block over raw on-disk bytes; (5) set reserved fields to zero; (6) pad8 all variable fields; (7) never modify written blocks or the file header after subsequent data is written; (8) set `CapBitmap` accurately; (9) if importing: set `Flags` bit 0 and emit Import Provenance (`0x0030`) as Block 0.
A conformant producer **SHOULD** emit an End-of-Capture block (type `0x0FFF`, Section 4.5) as the final capture-time block to close the integrity loop within the file.

## 12.2 Consumer

A conformant consumer **MUST**: (1) reject files with invalid file magic or `Endianness`; (2) use `HeaderSize` to locate Block 0; (3) skip unknown block types via `BlockLength`; (4) detect end-of-blocks when fewer than 80 bytes remain or the next 4 bytes are not `0x4D534C43`; (5) ignore reserved fields.
A conformant consumer **SHOULD**: (1) verify the BLAKE3 chain from the file header through all blocks; (2) if an EoC block is present, verify `FileHash` against the computed digest of the file up to the EoC block; (3) check `CapBitmap` before searching for specific block types. If no EoC block is present, the consumer **SHOULD** verify the file against an externally stored digest when available.

# References

[1] S. Bradner, "Key words for use in RFCs to Indicate Requirement Levels," RFC 2119, 1997. https://www.rfc-editor.org/rfc/rfc2119

[2] P. Leach, M. Mealling, R. Salz, "A Universally Unique IDentifier (UUID) URN Namespace," RFC 4122, 2005. https://www.rfc-editor.org/rfc/rfc4122

[3] J. O'Connor et al., "BLAKE3: One function, fast everywhere," 2020. https://github.com/BLAKE3-team/BLAKE3-specs/blob/master/blake3.pdf

[4] TIS Committee, "ELF Specification," v1.2, 1995.